

Customer Churn Prediction & Analytics System

Project Report

VITyarthi — Build Your Own Project
Course: Machine Learning / Artificial Intelligence
Date: September 18, 2026

Submitted as part of the flipped course evaluation, in accordance with the
General Project Instructions & Submission Guidelines.

2. Introduction

Customer churn — the loss of subscribers or paying customers to competitors or to inactivity — directly reduces recurring revenue for subscription-based businesses such as telecom operators, streaming services, and SaaS companies. Identifying customers who are likely to churn before they actually leave allows a business to intervene with targeted retention offers, which is significantly cheaper than acquiring a brand-new customer.

This project applies supervised machine learning concepts covered in the course to build a working, end-to-end **Customer Churn Prediction & Analytics System**. The system ingests customer account data, trains a classification model to estimate churn risk, exposes the model through a command-line interface, persists every prediction to a relational store with full CRUD support, and produces aggregate analytics over the stored history.

3. Problem Statement

Businesses with large customer bases cannot manually review every account to judge churn risk, and generic, un-targeted retention campaigns waste marketing budget on customers who were never going to leave. There is a need for an automated, data-driven system that can score individual customers on their likelihood to churn, and give retention teams an easy way to view both individual predictions and aggregate trends.

Scope

In scope: data ingestion & validation, preprocessing, churn classification model, a CLI for scoring customers on demand, persistent storage with full CRUD, and analytics reporting.

Out of scope: a web UI / REST API, real-time streaming ingestion, multi-user authentication, and automated MLOps retraining pipelines — noted in Section 14 as future enhancements.

Target Users

- Customer retention / marketing teams who want a ranked view of at-risk customers
- Data analysts who want quick churn analytics without writing ad-hoc queries
- Evaluators / students reviewing this project, who can run it locally with zero external dependencies

4. Functional Requirements

The system is organised into three major functional modules, satisfying the minimum of three required by the project brief:

#	Module	Responsibility	Input	Output
1	Data Ingestion & Preprocessing	Load, validate / clean / encode / scale customer data	Raw CSV (or synthetic generator)	Model-ready feature matrix
2	Model Training & Prediction	Train a classifier; score new customers	Training features, or one new customer	Trained model; churn label + probability
3	Storage & Reporting	Persist predictions (CRUD); generate analytics	Prediction results	Stored records; aggregate churn report

User Interaction Workflow

A user interacts through a numbered CLI menu (see Section 7's Workflow Diagram) to: (1) train or retrain the model, (2) predict churn for a new customer, (3) view the analytics report, and (4)–(6) read, update, or delete previously stored prediction records — a complete CRUD cycle over the prediction data.

5. Non-Functional Requirements

Requirement	How it is addressed in this project
Performance	Random Forest with bounded depth (<code>max_depth=8</code>) trains and predicts in under a second on the sample dataset; S
Reliability	Custom exceptions (<code>DataValidationError</code> , <code>PredictionError</code>) reject malformed input before it reaches the model, avoidi
Usability	A simple, numbered CLI menu with clearly labelled prompts and output requires no technical setup beyond installing
Maintainability	Single-responsibility modules (<code>data</code> , <code>preprocessing</code> , <code>model</code> , <code>predictor</code> , <code>database</code> , <code>report</code>) with all configuration centrali
Scalability	The prediction function is stateless per call; <code>database.py</code> isolates storage so SQLite can be swapped for PostgreSQL
Error handling strategy	Every module wraps risky operations (file I/O, DB access, model loading) in <code>try/except</code> blocks with specific, informati
Logging / Monitoring	A shared logger (<code>logger.py</code>) writes timestamped, leveled logs to both the console and <code>logs/app.log</code> for every module.
Security	All SQL statements in <code>database.py</code> are parameterised, preventing SQL injection from user-supplied values.

6. System Architecture

The system follows a layered, modular architecture. The CLI (Application Layer) is the only entry point users interact with; it delegates to three independent modules — Data, Model/Prediction, and Storage & Reporting — each of which owns a single responsibility. A shared Logger cuts across every layer for observability.

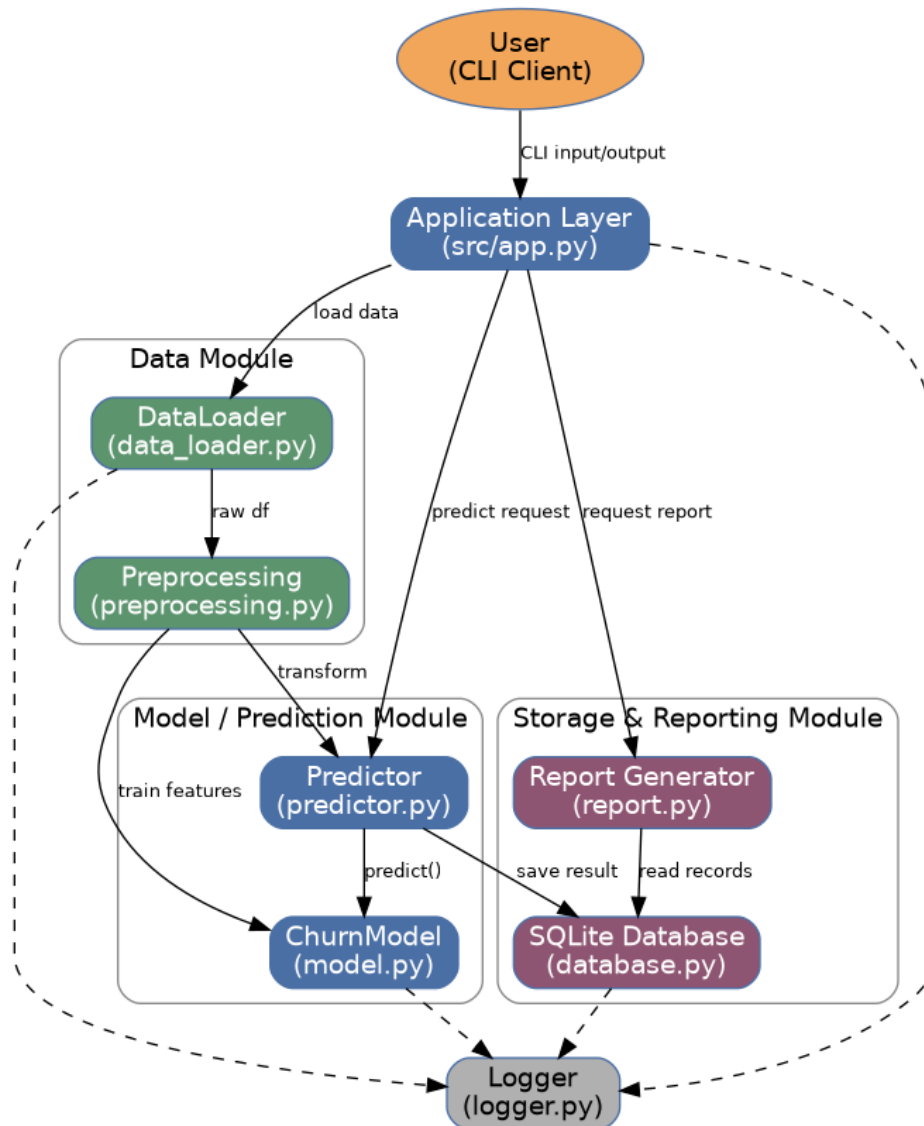


Figure 1: System Architecture Diagram

7. Design Diagrams

7.1 Use Case Diagram

The single actor (User) can trigger six use cases, mapping directly onto the CLI menu options.

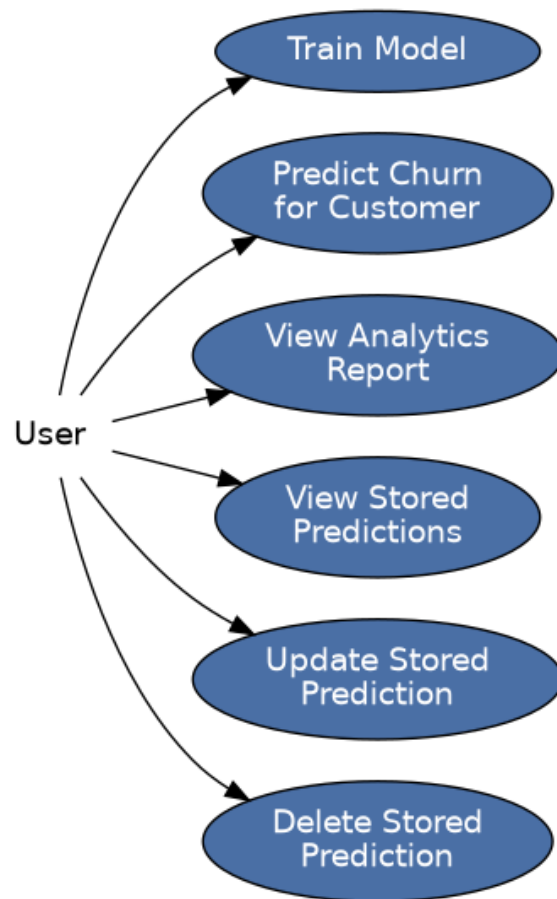


Figure 2: Use Case Diagram

7.2 Process / Workflow Diagram

Shows the full control flow of the CLI application, from menu display through each of the three main actions and back to the menu loop.

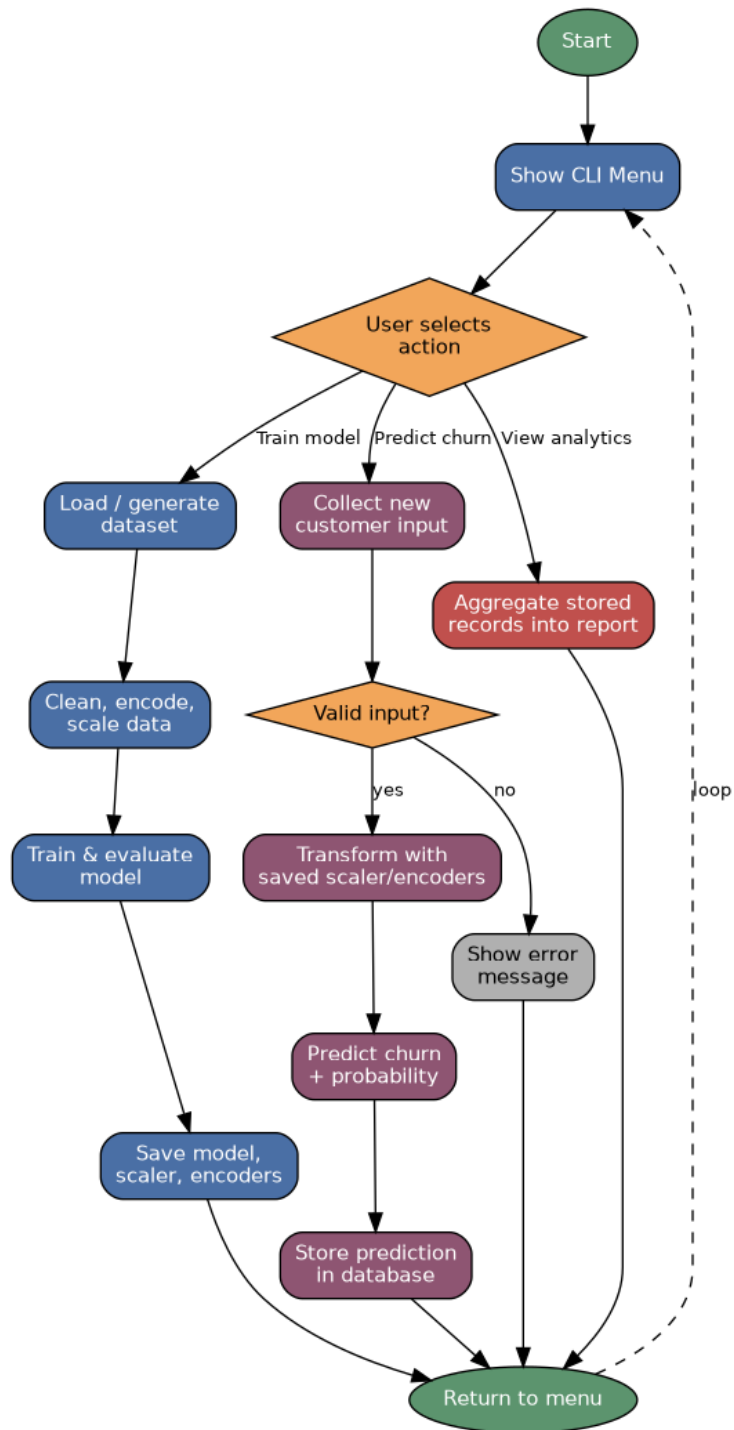


Figure 3: Process Flow / Workflow Diagram

7.3 Sequence Diagram — Predict Churn for a New Customer

Illustrates the call sequence for the most important use case: a user requesting a churn prediction for a new customer, from CLI input through preprocessing, model inference, and persistence back to the database.

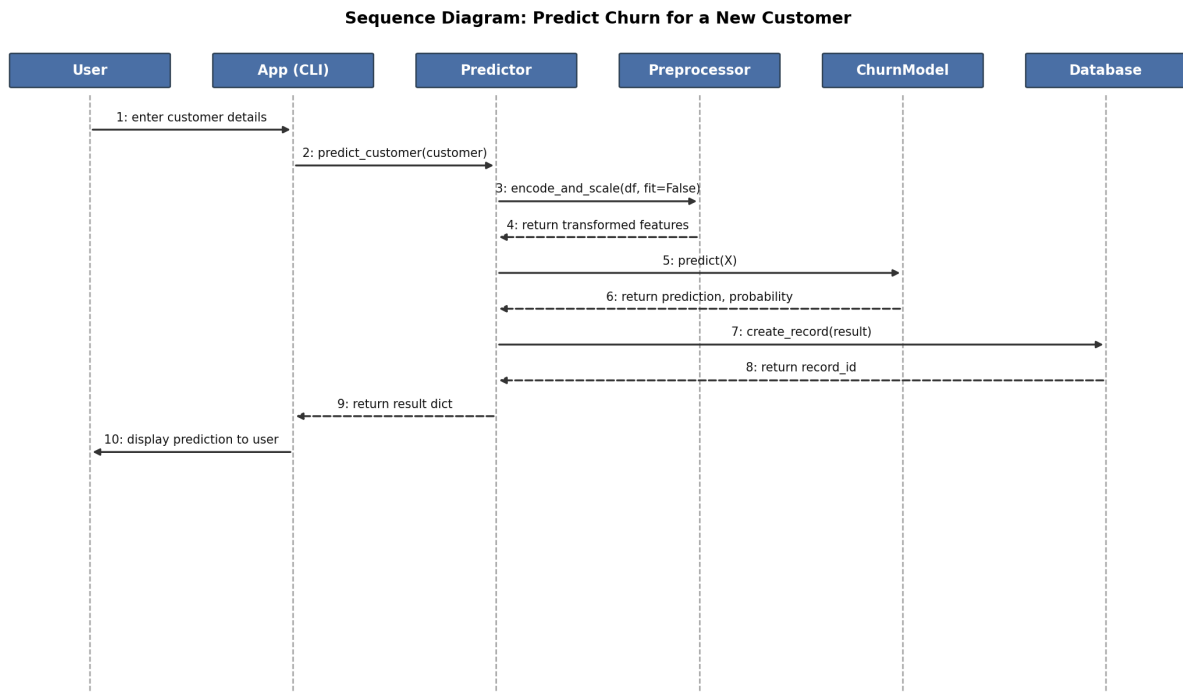


Figure 4: Sequence Diagram (Predict Churn Flow)

7.4 Class / Component Diagram

Shows the main classes/modules, their public interfaces, and their dependency relationships.

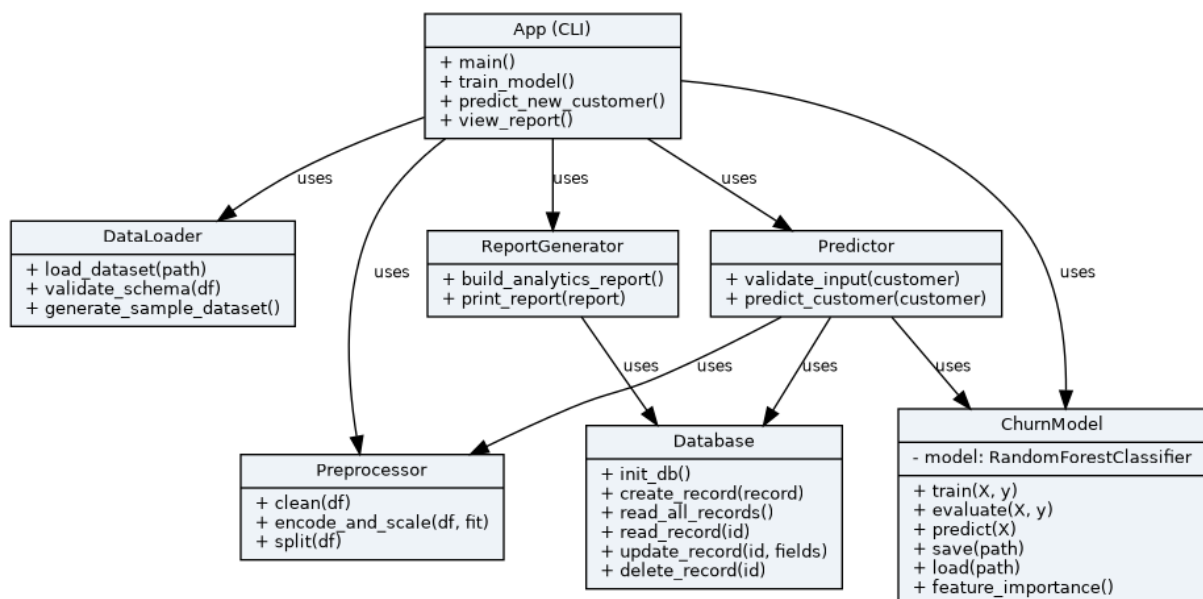


Figure 5: Class / Component Diagram

7.5 ER Diagram (Database Design)

The system uses a single, denormalised **predictions** table for simplicity, since a prediction record is self-contained (it stores the customer attributes used for that prediction alongside the outcome, so it doesn't depend on a separate, mutable customers table).

predictions
PK id : INTEGER (AUTOINCREMENT)
tenure_months : INTEGER
monthly_charges : REAL
total_charges : REAL
support_calls : INTEGER
contract_type : TEXT
internet_service : TEXT
payment_method : TEXT
churn_prediction : INTEGER
churn_probability : REAL
created_at : TEXT (ISO timestamp)

Figure 6: Entity-Relationship / Schema Diagram

8. Design Decisions & Rationale

- **Random Forest over a single decision tree or logistic regression:** Random Forest handles non-linear interactions between features (e.g. contract type combined with monthly charges) without heavy feature engineering, and is robust to the modest, mixed-type feature set used here.
- **SQLite over a client-server database:** the project needs zero external setup so it can be graded by simply cloning and running it; database.py isolates all SQL so swapping in PostgreSQL/MySQL later only requires changing the connection layer.
- **A single denormalised predictions table:** avoids the complexity of a customers/predictions join for a project of this scope, while still supporting full CRUD and analytics grouping.
- **Synthetic data generator built into data_loader.py:** guarantees the project runs end-to-end without needing an external dataset download, while still encoding a realistic churn signal (month-to-month contracts, high monthly charges, and frequent support calls raise churn probability) so the model has something genuine to learn.
- **CLI over a GUI/web app:** keeps the project's dependency footprint minimal and focuses the implementation effort on the machine-learning and software-engineering concepts being evaluated, rather than front-end work; a web dashboard is listed as a future enhancement.
- **Saved encoders/scaler as separate artefacts from the model:** ensures new data at prediction time is transformed identically to the training data, which is essential for correct inference and is a common source of production bugs when skipped.

9. Implementation Details

The project is implemented in Python 3 across 9 source modules plus 3 test modules (12 files total in src/ and tests/), organised as follows:

Module	Key Responsibilities
config.py	Central paths, column lists, and hyperparameters (RANDOM_STATE, TEST_SIZE).
logger.py	Shared logger writing to console and logs/app.log.
data_loader.py	CSV loading, schema validation, synthetic dataset generation.
preprocessing.py	Duplicate/missing-value cleaning, LabelEncoding, StandardScaler, train/test split.
model.py	ChurnModel wrapper around RandomForestClassifier: train, evaluate, save, load, feature_importance.
predictor.py	Validates and scores a single new customer, reusing saved encoders/scaler.
database.py	SQLite schema + full CRUD (create_record, read_all_records, read_record, update_record, delete_record).
report.py	Aggregates stored records into churn-rate and spending analytics.
app.py	CLI entry point wiring together all modules behind a numbered menu.

Key Snippet: Prediction Pipeline (predictor.py)

```
def predict_customer(customer: dict, persist: bool = True) -> dict:
    validate_input(customer)
    df = pd.DataFrame([customer])
```

```
transformed = encode_and_scale(df, fit=False) # reuse saved scaler/encoders
model = ChurnModel().load()
X = transformed[NUMERIC_COLUMNS + CATEGORICAL_COLUMNS]
preds, probs = model.predict(X)
result = {..., "churn_prediction": int(preds[0]),
              "churn_probability": round(float(probs[0]), 4)}
if persist:
    database.create_record(result)
return result
```

10. Screenshots / Results

As this is a CLI-based system, representative terminal output is shown below in place of GUI screenshots, captured from an actual run of the trained pipeline.

Model Training Output

```
Model trained successfully.
Evaluation metrics: {'accuracy': 0.79, 'precision': 0.7917, 'recall': 0.5429, 'f1_score':
0.6441}
Top features: [('contract_type', 0.2533), ('monthly_charges', 0.2113), ('tenure_months',
0.1713)]
```

Sample Predictions

```
Customer A (2 months tenure, month-to-month, $95/mo, 4 support calls):
-> LIKELY TO CHURN (probability = 0.9792)

Customer B (60 months tenure, two-year contract, $45/mo, 0 support calls):
-> LIKELY TO STAY (probability = 0.0058)

Customer C (20 months tenure, one-year contract, $70/mo, 1 support call):
-> LIKELY TO STAY (probability = 0.0462)
```

Analytics Report Output

```
--- Churn Analytics Report ---
Total records : 3
Predicted churners : 1
Predicted churn rate : 33.33%
Avg churn probability : 0.3437
Churn rate by contract type:
Month-to-month : 100.00%
One year : 0.00%
Two year : 0.00%
Avg monthly charges (0=stay, 1=churn):
0: 57.5
1: 95.0
```

11. Testing Approach

Unit tests are provided for each of the three core modules, using plain **assert** statements (runnable directly with python, no external test framework required):

Test file	What it validates
tests/test_preprocessing.py	Duplicate rows are removed; categorical columns become numeric after encoding; train/test split preserves
tests/test_model.py	The model trains without error and its accuracy is at least close to the naive majority-class baseline; predict
tests/test_database.py	Full CRUD cycle on a temporary SQLite file: create returns a valid id, read returns the inserted record, up

All three test files were executed during development and passed: All preprocessing tests passed. / All model tests passed. / All database tests passed.

An end-to-end smoke test (train → predict → report) was also run manually to confirm the modules integrate correctly; its output is shown in Section 10.

12. Challenges Faced

- **Consistent train/predict transformation:** early on, encoding new customer data with a freshly-fit encoder produced different category codes than the ones the model was trained on. Solved by persisting the fitted LabelEncoders and StandardScaler with joblib and reusing them (fit=False) at prediction time.
- **Class imbalance:** the synthetic churn label is imbalanced (roughly 60/40), which can bias a classifier toward the majority class. Mitigated using class_weight='balanced' in the RandomForestClassifier so both classes contribute proportionally to training.
- **Keeping the project runnable without an external dataset:** real churn datasets aren't bundled with the assignment, so a synthetic-but-realistic data generator was built with a genuine underlying churn signal, rather than fully random labels, so the trained model performs meaningfully better than chance.
- **Rendering UML-style diagrams programmatically:** Graphviz does not have native sequence-diagram notation, so the sequence diagram was instead generated with matplotlib using explicit lifelines and staggered arrows to keep the labels readable.

13. Learnings & Key Takeaways

- End-to-end ML systems need more than a trained model — reproducible preprocessing, persistence, and a serving interface are just as important as model accuracy.
- Centralising configuration and logging early made the rest of the modules noticeably easier to debug and keep consistent.
- Designing the database schema before writing the CRUD layer made the storage module far simpler to implement correctly the first time.
- Small, single-responsibility modules were easier to unit test in isolation than a monolithic script would have been.

14. Future Enhancements

- Replace the CLI with a web dashboard (Flask or Streamlit) for non-technical users
- Support bulk CSV upload for batch scoring of many customers at once
- Add model versioning and side-by-side comparison of different algorithms (e.g. Gradient Boosting, Logistic Regression)
- Add authentication and role-based access for multi-user / multi-team deployments
- Automate periodic retraining as new labelled outcomes become available (MLOps pipeline)

15. References

- scikit-learn documentation — RandomForestClassifier, StandardScaler, LabelEncoder — scikit-learn.org
- pandas documentation — pandas.pydata.org
- Python sqlite3 module documentation — docs.python.org/3/library/sqlite3.html
- Python logging module documentation — docs.python.org/3/library/logging.html
- Graphviz documentation — graphviz.org